Neural Network

Load libraries

```
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.
0.0 ──
## ✓ dplyr     1.2.1     ✓ readr     2.2.0
## ✓ forcats   1.0.1     ✓ stringr   1.6.0
## ✓ ggplot2   4.0.3     ✓ tibble    3.3.1
## ✓ lubridate 1.9.5     ✓ tidyr     1.3.2
## ✓ purrr     1.2.2
## ── Conflicts ────────────────────────────────────────── tidyverse_conflict
s() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
 conflicts to become errors
```

```
library(tidymodels)
```

```
## ── Attaching packages ─────────────────────────────────────── tidymodels 1.
5.0 ──
## ✓ broom        1.0.12     ✓ rsample      1.3.2
## ✓ dials        1.4.3      ✓ tailor       0.1.0
## ✓ infer        1.1.0      ✓ tune         2.1.0
## ✓ modeldata    1.5.1      ✓ workflows    1.3.0
## ✓ parsnip      1.6.0      ✓ workflowsets 1.1.1
## ✓ recipes      1.3.2      ✓ yardstick    1.4.0
## ── Conflicts ────────────────────────────────────── tidymodels_conflict
s() ──
## ✖ scales::discard() masks purrr::discard()
## ✖ dplyr::filter()   masks stats::filter()
## ✖ recipes::fixed()  masks stringr::fixed()
## ✖ dplyr::lag()      masks stats::lag()
## ✖ yardstick::spec() masks readr::spec()
## ✖ recipes::step()   masks stats::step()
```

```
library(mice)
```

```
##
## Attaching package: 'mice'
##
## The following object is masked from 'package:stats':
##
##     filter
```

```
##
## The following objects are masked from 'package:base':
##
##     cbind, rbind
```

```
library(VIM)
```

```
## Loading required package: colorspace
## Loading required package: grid
## Registered S3 method overwritten by 'car':
##   method           from
##   na.action.merMod lme4
## VIM is ready to use.
##
## Suggestions and bug-reports can be submitted at: https://github.com/statis
tikat/VIM/issues
##
## Attaching package: 'VIM'
##
## The following object is masked from 'package:recipes':
##
##     prepare
##
## The following object is masked from 'package:datasets':
##
##     sleep
```

```
library(ranger)
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following objects are masked from 'package:yardstick':
##
##     precision, recall, sensitivity, specificity
##
## The following object is masked from 'package:rsample':
##
##     calibration
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(skimr)
library(RColorBrewer)
library(rpart)
```

```
##
## Attaching package: 'rpart'
##
## The following object is masked from 'package:dials':
##
##     prune
```

```
library(rattle)
```

```
## Loading required package: bitops
## Rattle: A free graphical interface for data science with R.
## Version 5.6.2 Copyright (c) 2006-2023 Togaware Pty Ltd.
## Type 'rattle()' to shake, rattle, and roll your data.
##
## Attaching package: 'rattle'
##
## The following object is masked from 'package:ranger':
##
##     importance
##
## The following object is masked from 'package:VIM':
##
##     wine
```

```
library(e1071)
```

```
##
## Attaching package: 'e1071'
##
## The following object is masked from 'package:tune':
##
##     tune
##
## The following object is masked from 'package:rsample':
##
##     permutations
##
## The following object is masked from 'package:parsnip':
##
##     tune
##
## The following object is masked from 'package:ggplot2':
##
##     element
```

```
library(xgboost)
```

```
##
## Attaching package: 'xgboost'
##
## The following object is masked from 'package:rattle':
```

```
##
##     xgboost

library(usemodels)
library(nnet)
```

Read in dataset

```
train <- read_csv("C:/Users/miche/OneDrive/Desktop/BAN 502 Predictive Analytics/Course Project/train.csv")

## Rows: 26570 Columns: 26
## ── Column specification ────────────────────────────────────────────────────
## Delimiter: ","
## chr  (4): product_code, attribute_0, attribute_1, failure
## dbl (22): id, loading, attribute_2, attribute_3, measurement_0, measurement_...
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Structure and summary

```
str(train)

## spc_tbl_ [26,570 × 26] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
##  $ id             : num [1:26570] 0 1 2 3 4 5 6 7 8 9 ...
##  $ product_code   : chr [1:26570] "A" "A" "A" "A" ...
##  $ loading        : num [1:26570] 80.1 84.9 82.4 101.1 188.1 ...
##  $ attribute_0    : chr [1:26570] "material_7" "material_7" "material_7" "material_7" ...
##  $ attribute_1    : chr [1:26570] "material_8" "material_8" "material_8" "material_8" ...
##  $ attribute_2    : num [1:26570] 9 9 9 9 9 9 9 9 9 9 ...
##  $ attribute_3    : num [1:26570] 5 5 5 5 5 5 5 5 5 5 ...
##  $ measurement_0  : num [1:26570] 7 14 12 13 9 11 12 4 9 10 ...
##  $ measurement_1  : num [1:26570] 8 3 1 2 2 4 2 8 6 4 ...
##  $ measurement_2  : num [1:26570] 4 3 5 6 8 0 4 8 5 7 ...
##  $ measurement_3  : num [1:26570] 18 18.2 18.1 17.3 19.3 ...
##  $ measurement_4  : num [1:26570] 12.5 11.5 11.7 11.2 12.9 ...
##  $ measurement_5  : num [1:26570] 15.7 17.7 16.7 18.6 17 ...
##  $ measurement_6  : num [1:26570] 19.3 17.9 18.2 18.3 15.7 ...
##  $ measurement_7  : num [1:26570] 11.7 12.7 12.7 12.6 11.3 ...
##  $ measurement_8  : num [1:26570] 20.2 17.9 18.3 19.1 18.1 ...
##  $ measurement_9  : num [1:26570] 10.7 12.4 12.7 12.5 10.3 ...
##  $ measurement_10 : num [1:26570] 15.9 17.9 15.6 16.3 17.1 ...
##  $ measurement_11 : num [1:26570] 17.6 17.9 NA 18.4 19.9 ...
##  $ measurement_12 : num [1:26570] 15.2 11.8 13.8 10 12.4 ...
##  $ measurement_13 : num [1:26570] 15 14.7 16.7 15.2 16.2 ...
```

```
##  $ measurement_14: num [1:26570] NA 15.4 18.6 15.6 12.8 ...
##  $ measurement_15: num [1:26570] 13 14.4 14.1 16.2 13.2 ...
##  $ measurement_16: num [1:26570] 14.7 15.6 17.9 17.2 16.4 ...
##  $ measurement_17: num [1:26570] 764 682 663 826 580 ...
##  $ failure       : chr [1:26570] "No" "No" "No" "No" ...
##  - attr(*, "spec")=
##   .. cols(
##   ..   id = col_double(),
##   ..   product_code = col_character(),
##   ..   loading = col_double(),
##   ..   attribute_0 = col_character(),
##   ..   attribute_1 = col_character(),
##   ..   attribute_2 = col_double(),
##   ..   attribute_3 = col_double(),
##   ..   measurement_0 = col_double(),
##   ..   measurement_1 = col_double(),
##   ..   measurement_2 = col_double(),
##   ..   measurement_3 = col_double(),
##   ..   measurement_4 = col_double(),
##   ..   measurement_5 = col_double(),
##   ..   measurement_6 = col_double(),
##   ..   measurement_7 = col_double(),
##   ..   measurement_8 = col_double(),
##   ..   measurement_9 = col_double(),
##   ..   measurement_10 = col_double(),
##   ..   measurement_11 = col_double(),
##   ..   measurement_12 = col_double(),
##   ..   measurement_13 = col_double(),
##   ..   measurement_14 = col_double(),
##   ..   measurement_15 = col_double(),
##   ..   measurement_16 = col_double(),
##   ..   measurement_17 = col_double(),
##   ..   failure = col_character()
##   .. )
##  - attr(*, "problems")=<pointer: 0x000001c0550fe9e0>
```

```
summary(train)
```

```
##        id           product_code      loading          attribute_0
##  Min.   :    0   Length   :26570   Min.   : 33.16   Length   :26570
##  1st Qu.: 6642   N.unique :    5   1st Qu.: 99.99   N.unique :    2
##  Median :13284   N.blank  :    0   Median :122.39   N.blank  :    0
##  Mean   :13284   Min.nchar:    1   Mean   :127.83   Min.nchar:   10
##  3rd Qu.:19927   Max.nchar:    1   3rd Qu.:149.15   Max.nchar:   10
##  Max.   :26569                     Max.   :385.86
##                                    NAs    :250
##     attribute_1     attribute_2     attribute_3   measurement_0
##  Length   :26570   Min.   :5.000   Min.   :5.00   Min.   : 0.000
##  N.unique :    3   1st Qu.:6.000   1st Qu.:6.00   1st Qu.: 4.000
##  N.blank  :    0   Median :6.000   Median :8.00   Median : 7.000
```

```
##  Min.nchar:   10   Mean   :6.754   Mean   :7.24   Mean   : 7.416
##  Max.nchar:   10   3rd Qu.:8.000   3rd Qu.:8.00   3rd Qu.:10.000
##                    Max.   :9.000   Max.   :9.00   Max.   :29.000
##
##  measurement_1    measurement_2    measurement_3   measurement_4
##  Min.   : 0.000   Min.   : 0.000   Min.   :13.97   Min.   : 8.008
##  1st Qu.: 5.000   1st Qu.: 4.000   1st Qu.:17.12   1st Qu.:11.051
##  Median : 8.000   Median : 6.000   Median :17.79   Median :11.733
##  Mean   : 8.233   Mean   : 6.257   Mean   :17.79   Mean   :11.732
##  3rd Qu.:11.000   3rd Qu.: 8.000   3rd Qu.:18.47   3rd Qu.:12.410
##  Max.   :29.000   Max.   :24.000   Max.   :21.50   Max.   :16.484
##                                    NAs    :381     NAs    :538
##  measurement_5   measurement_6   measurement_7    measurement_8
##  Min.   :12.07   Min.   :12.71   Min.   : 7.968   Min.   :15.22
##  1st Qu.:16.44   1st Qu.:16.84   1st Qu.:11.045   1st Qu.:18.34
##  Median :17.13   Median :17.52   Median :11.712   Median :19.02
##  Mean   :17.13   Mean   :17.51   Mean   :11.717   Mean   :19.02
##  3rd Qu.:17.80   3rd Qu.:18.18   3rd Qu.:12.391   3rd Qu.:19.71
##  Max.   :21.43   Max.   :21.54   Max.   :15.419   Max.   :23.81
##  NAs    :676     NAs    :796     NAs    :937     NAs    :1048
##  measurement_9    measurement_10   measurement_11  measurement_12
##  Min.   : 7.537   Min.   : 9.323   Min.   :12.46   Min.   : 5.167
##  1st Qu.:10.757   1st Qu.:15.209   1st Qu.:18.17   1st Qu.:10.703
##  Median :11.430   Median :16.127   Median :19.21   Median :11.717
##  Mean   :11.431   Mean   :16.118   Mean   :19.17   Mean   :11.702
##  3rd Qu.:12.102   3rd Qu.:17.025   3rd Qu.:20.21   3rd Qu.:12.709
##  Max.   :15.412   Max.   :22.479   Max.   :25.64   Max.   :17.663
##  NAs    :1227     NAs    :1300     NAs    :1468    NAs    :1601
##  measurement_13  measurement_14  measurement_15   measurement_16
##  Min.   :10.89   Min.   : 9.14   Min.   : 9.104   Min.   : 9.701
##  1st Qu.:14.89   1st Qu.:15.06   1st Qu.:13.957   1st Qu.:15.268
##  Median :15.63   Median :16.04   Median :14.969   Median :16.436
##  Mean   :15.65   Mean   :16.05   Mean   :14.996   Mean   :16.461
##  3rd Qu.:16.37   3rd Qu.:17.08   3rd Qu.:16.018   3rd Qu.:17.628
##  Max.   :22.71   Max.   :22.30   Max.   :21.626   Max.   :24.094
##  NAs    :1774    NAs    :1874    NAs    :2009     NAs    :2110
##  measurement_17        failure
##  Min.   : 196.8   Length   :26570
##  1st Qu.: 619.0   N.unique :    2
##  Median : 701.0   N.blank  :    0
##  Mean   : 701.3   Min.nchar:    2
##  3rd Qu.: 784.1   Max.nchar:    3
##  Max.   :1312.8
##  NAs    :2284
```

Delete ID column

```
train = train %>% select(- id)
```

Convert character variables to factors

```
train = train %>% mutate_if(is.character, as_factor)
```

Convert other categorical variables to factors

```
train <- train %>%
  mutate(
    attribute_2 = as.factor(attribute_2),
    attribute_3 = as.factor(attribute_3)
  )
```

Clean data, remove outliers

```
train = train %>% filter(measurement_2 < 22)
train = train %>% filter(measurement_8 < 23.5)
train = train %>% filter(measurement_14 > 8)
train = train %>% filter(measurement_15 < 23)
train = train %>% filter(measurement_17 < 1500)
```

Select variables of interest

```
train = train %>% select(c("loading", "measurement_17", "measurement_2", "mea
surement_5", "measurement_6", "measurement_8", "failure"))
```

Check for missing data

```
skim(train)
```

Data summary

Name	train
Number of rows	20043
Number of columns	7
_____	
Column type frequency:	
factor	1
numeric	6
_____	
Group variables	None

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
failure	0	1	FALSE	2	No: 15803, Yes: 4240

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
loading	178	0.99	127.88	38.94	40.81	100.16	122.48	149.15	385.86	
measurement_17	0	1.00	701.26	123.05	196.79	618.64	700.76	783.41	1312.79	
measurement_2	0	1.00	6.25	3.30	0.00	4.00	6.00	8.00	21.00	
measurement_5	501	0.98	17.13	1.00	12.07	16.45	17.13	17.81	21.42	
measurement_6	599	0.97	17.51	1.00	12.71	16.83	17.52	18.17	21.54	
measurement_8	0	1.00	19.02	1.01	15.27	18.34	19.02	19.71	23.33	

Imputation

```
set.seed(1234)
imp_train = mice(train, m=5, method="pmm", printFlag=FALSE)

train_complete = complete(imp_train)
str(train_complete)
```

```
## 'data.frame':    20043 obs. of  7 variables:
##  $ loading       : num  84.9 82.4 101.1 188.1 177.9 ...
##  $ measurement_17: num  682 663 826 580 684 ...
##  $ measurement_2 : num  3 5 6 8 8 6 2 6 10 10 ...
##  $ measurement_5 : num  17.7 16.7 18.6 17 17.9 ...
##  $ measurement_6 : num  17.9 18.2 18.3 15.7 15.9 ...
##  $ measurement_8 : num  17.9 18.3 19.1 18.1 18.6 ...
##  $ failure       : Factor w/ 2 levels "No","Yes": 1 1 1 1 2 1 2 1 2 1 ...
```

```
summary(train_complete)
```

```
##     loading       measurement_17   measurement_2    measurement_5
##  Min.   : 40.81   Min.   : 196.8   Min.   : 0.000   Min.   :12.07
##  1st Qu.:100.13   1st Qu.: 618.6   1st Qu.: 4.000   1st Qu.:16.45
##  Median :122.46   Median : 700.8   Median : 6.000   Median :17.13
##  Mean   :127.84   Mean   : 701.3   Mean   : 6.251   Mean   :17.13
##  3rd Qu.:149.07   3rd Qu.: 783.4   3rd Qu.: 8.000   3rd Qu.:17.81
##  Max.   :385.86   Max.   :1312.8   Max.   :21.000   Max.   :21.43
##  measurement_6   measurement_8   failure
##  Min.   :12.71   Min.   :15.27   No :15803
##  1st Qu.:16.83   1st Qu.:18.34   Yes: 4240
##  Median :17.52   Median :19.02
##  Mean   :17.51   Mean   :19.02
##  3rd Qu.:18.17   3rd Qu.:19.71
##  Max.   :21.54   Max.   :23.33
```

Split the data

```
set.seed(123)
train_split = initial_split(train_complete, prop=0.7, strata=failure)
train_set = training(train_split)
test_set = testing(train_split)
```

Set up folds

```
set.seed(123)
folds = vfold_cv(train_set, v = 5)
```

Neural network w/ R tuning

```
start_time = Sys.time()

train_recipe = recipe(failure ~ loading, train_set) %>%
  step_dummy(all_nominal(), -all_outcomes()) %>%
  step_normalize(all_numeric_predictors())

train_model =
  mlp(hidden_units = tune(), penalty = tune(),
      epochs = tune()) %>%
  set_mode("classification") %>%
  set_engine("nnet", verbose = FALSE)
```

```
train_workflow =
  workflow() %>%
  add_recipe(train_recipe) %>%
  add_model(train_model)

set.seed(1234)
neural_tune =
  tune_grid(train_workflow, resamples = folds, grid = 25)

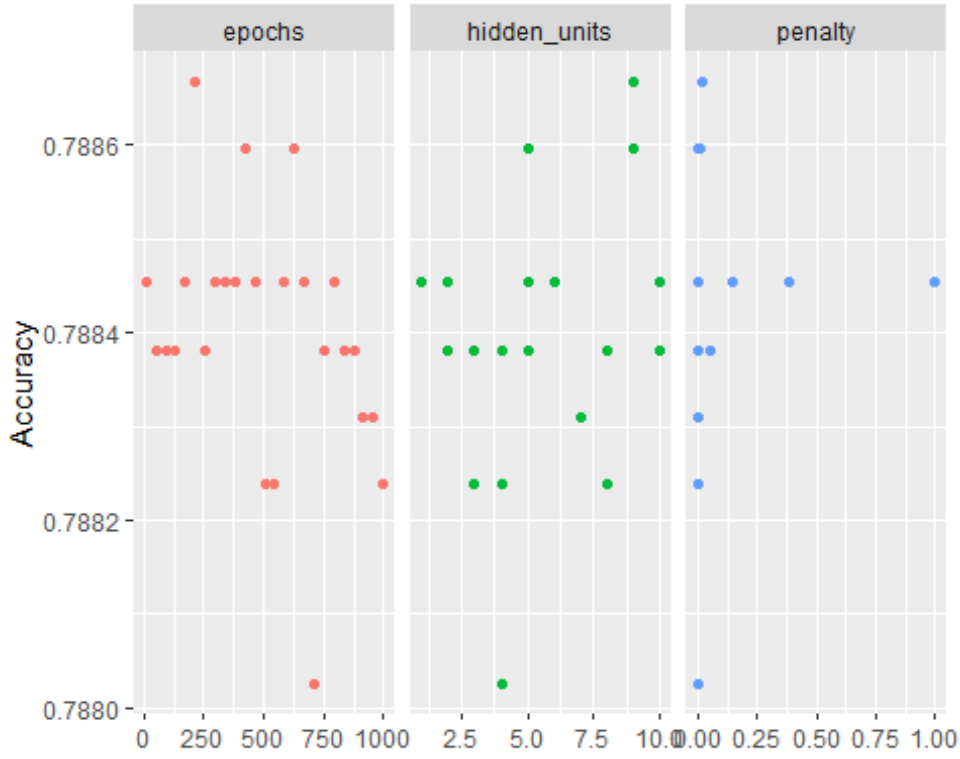
end_time = Sys.time()
end_time - start_time

## Time difference of 2.364458 mins
```

Visualization of tune parameters

```
neural_tune %>%
  collect_metrics() %>%
  filter(.metric == "accuracy") %>%
  select(mean, hidden_units, penalty, epochs) %>%
  pivot_longer(
    hidden_units:epochs,
                values_to = "values",
                names_to = "parameter"
                ) %>%

  ggplot(aes(values, mean, color = parameter)) +
  geom_point(show.legend = FALSE) +
  facet_wrap(~parameter, scales = "free_x") +
  labs(x = NULL, y = "Accuracy")
```



Obtain best parameter values

```
best_nn <- tune::select_best(neural_tune, metric = "accuracy")

final_nn <- finalize_workflow(
  train_workflow,
  best_nn
)

final_nn
```

```
## == Workflow ====================================================================
======
## Preprocessor: Recipe
## Model: mlp()
##
## -- Preprocessor ----------------------------------------------------------------
------
## 2 Recipe Steps
##
## • step_dummy()
## • step_normalize()
##
## -- Model ----------------------------------------------------------------------
------
## Single Layer Neural Network Model Specification (classification)
##
```

```
## Main Arguments:
##   hidden_units = 9
##   penalty = 0.0215443469003189
##   epochs = 216
##
## Engine-Specific Arguments:
##   verbose = FALSE
##
## Computational engine: nnet
```

Fit model to train_set

```
final_nn_fit = fit(final_nn, train_set)
```

Predictions on train_set

```
trainprednn = predict(final_nn_fit, train_set)
head(trainprednn)
```

```
## # A tibble: 6 × 1
##   .pred_class
##   <fct>
## 1 No
## 2 No
## 3 No
## 4 No
## 5 No
## 6 No
```

Confusion matrix for train_set predictions

```
confusionMatrix(trainprednn$.pred_class, train_set$failure,
                positive = "Yes")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    No   Yes
##        No  11058  2962
##        Yes     4     6
##
##                Accuracy : 0.7886
##                  95% CI : (0.7817, 0.7953)
##     No Information Rate : 0.7885
##     P-Value [Acc > NIR] : 0.4884
##
##                   Kappa : 0.0026
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.0020216
```

```
##             Specificity : 0.9996384
##          Pos Pred Value : 0.6000000
##          Neg Pred Value : 0.7887304
##              Prevalence : 0.2115467
##          Detection Rate : 0.0004277
##    Detection Prevalence : 0.0007128
##       Balanced Accuracy : 0.5008300
##
##        'Positive' Class : Yes
##
```

Predictions on the test_set

```
testprednn = predict(final_nn_fit, test_set)
head(testprednn)
```

```
## # A tibble: 6 × 1
##   .pred_class
##   <fct>
## 1 No
## 2 No
## 3 No
## 4 No
## 5 No
## 6 No
```

Confusion matrix for test_set predictions

```
confusionMatrix(testprednn$.pred_class, test_set$failure,
                positive = "Yes")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   No  Yes
##        No  4737 1271
##        Yes    4    1
##
##                Accuracy : 0.788
##                  95% CI : (0.7774, 0.7982)
##     No Information Rate : 0.7885
##     P-Value [Acc > NIR] : 0.5452
##
##                   Kappa : -1e-04
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.0007862
##             Specificity : 0.9991563
##          Pos Pred Value : 0.2000000
##          Neg Pred Value : 0.7884487
```

```
##              Prevalence : 0.2115417
##          Detection Rate : 0.0001663
##    Detection Prevalence : 0.0008315
##       Balanced Accuracy : 0.4999712
##
##        'Positive' Class : Yes
##
```

Neural network manual tuning

```
start_time = Sys.time()

neural_grid = grid_regular(
  hidden_units(range = c(1,4)),
  penalty(range = c(-5, 0)),
  epochs(range = c(100,900)),
  levels = 10
)

train_recipe = recipe(failure ~ loading, train_set) %>%
  # step_dummy(all_nominal(), -all_outcomes()) %>%
  step_normalize(all_numeric_predictors())

train_model =
  mlp(hidden_units = tune(), penalty = tune(),
      epochs = tune()) %>%
  set_mode("classification") %>%
  set_engine("nnet", verbose = FALSE)

train_workflow =
  workflow() %>%
  add_recipe(train_recipe) %>%
  add_model(train_model)

set.seed(1234)
neural_tune =
  tune_grid(train_workflow, resamples = folds, grid = neural_grid)

end_time = Sys.time()
end_time - start_time

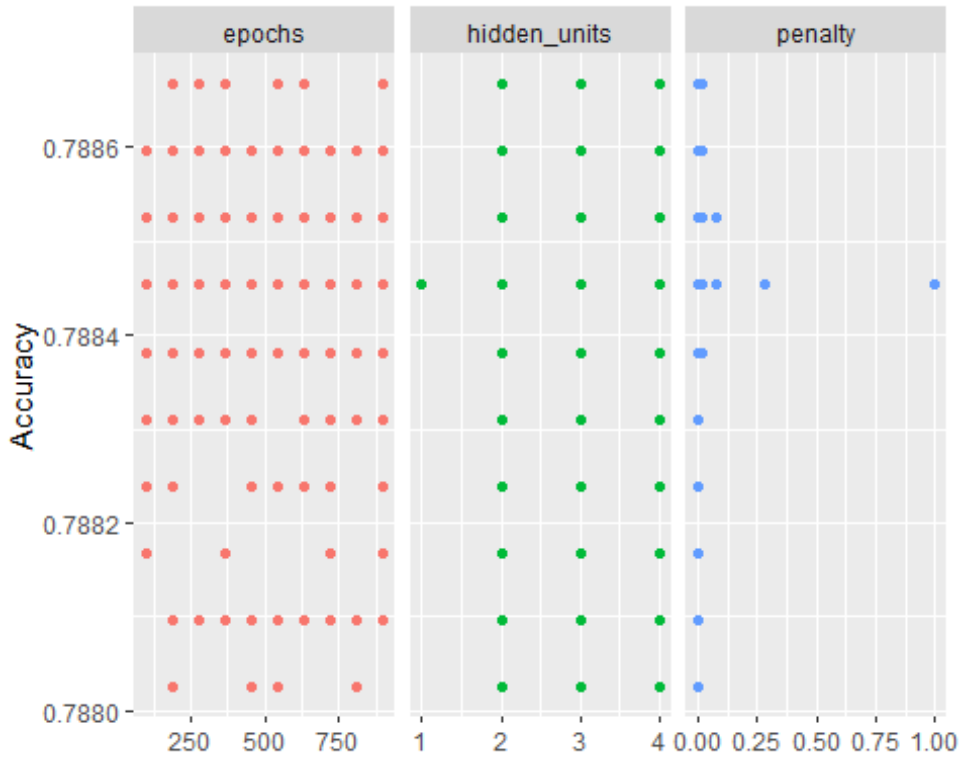
## Time difference of 19.83344 mins
```

Visualization of manual tune parameters

```
neural_tune %>%
  collect_metrics() %>%
  filter(.metric == "accuracy") %>%
  select(mean, hidden_units, penalty, epochs) %>%
  pivot_longer(
```

```
    hidden_units:epochs,
                values_to = "values",
                names_to = "parameter"
                ) %>%

  ggplot(aes(values, mean, color = parameter)) +
  geom_point(show.legend = FALSE) +
  facet_wrap(~parameter, scales = "free_x") +
  labs(x = NULL, y = "Accuracy")
```



Obtain best tune parameters

```
best_nn <- tune::select_best(neural_tune, metric = "accuracy")

final_nn <- finalize_workflow(
  train_workflow,
  best_nn
)

final_nn
```

```
## == Workflow ====================================================================
======
## Preprocessor: Recipe
## Model: mlp()
##
## -- Preprocessor ----------------------------------------------------------------
```

```
──────
## 1 Recipe Step
##
## • step_normalize()
##
## ── Model ────────────────────────────────────────────────────────────────────
──────
## Single Layer Neural Network Model Specification (classification)
##
## Main Arguments:
##   hidden_units = 2
##   penalty = 0.000464158883361278
##   epochs = 366
##
## Engine-Specific Arguments:
##   verbose = FALSE
##
## Computational engine: nnet
```

Fit the final workflow to train_set

```
final_nn_fit = fit(final_nn, train_set)
```

Predictions for train_set

```
trainprednn = predict(final_nn_fit, train_set)
head(trainprednn)
```

```
## # A tibble: 6 × 1
##   .pred_class
##   <fct>
## 1 No
## 2 No
## 3 No
## 4 No
## 5 No
## 6 No
```

Confusion matric on train_set

```
confusionMatrix(trainprednn$.pred_class, train_set$failure,
                positive = "Yes")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    No   Yes
##        No  11058  2962
##        Yes     4     6
##
##                Accuracy : 0.7886
```

```
##                  95% CI : (0.7817, 0.7953)
##     No Information Rate : 0.7885
##     P-Value [Acc > NIR] : 0.4884
##
##                   Kappa : 0.0026
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.0020216
##             Specificity : 0.9996384
##          Pos Pred Value : 0.6000000
##          Neg Pred Value : 0.7887304
##              Prevalence : 0.2115467
##          Detection Rate : 0.0004277
##    Detection Prevalence : 0.0007128
##       Balanced Accuracy : 0.5008300
##
##        'Positive' Class : Yes
##
```

Predictions test_set

```
testprednn = predict(final_nn_fit, test_set)
head(testprednn)
```

```
## # A tibble: 6 × 1
##   .pred_class
##   <fct>
## 1 No
## 2 No
## 3 No
## 4 No
## 5 No
## 6 No
```

Confusion matric on test_set

```
confusionMatrix(testprednn$.pred_class, test_set$failure,
                positive = "Yes")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   No  Yes
##        No  4737 1272
##        Yes    4    0
##
##                Accuracy : 0.7878
##                  95% CI : (0.7772, 0.7981)
##     No Information Rate : 0.7885
##     P-Value [Acc > NIR] : 0.5577
```

```
##
##                   Kappa : -0.0013
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.0000000
##             Specificity : 0.9991563
##          Pos Pred Value : 0.0000000
##          Neg Pred Value : 0.7883175
##              Prevalence : 0.2115417
##          Detection Rate : 0.0000000
##    Detection Prevalence : 0.0006652
##       Balanced Accuracy : 0.4995781
##
##        'Positive' Class : Yes
##
```